

# 智能应用系统开发方法研究——以 ShinobuChat 智能桌面伴侣为例

06096911 智能应用系统开发

学号-姓名：202306120101-蔡铭溢

## 摘要

本文以 ShinobuChat 智能桌面伴侣项目为案例，研究 AI-Native 智能应用系统的设计思想与开发方法。论文从普通系统与智能系统的范式差异出发，分析自然语言交互、概率性推理、数据一体化、长期记忆、工具调用和主动服务对软件架构的影响。

# 第一章 引言

本课程“智能应用系统开发”的核心目标，是使学习者掌握以大语言模型为核心驱动力的智能应用系统设计思想与开发方法。传统软件开发通常围绕确定性逻辑展开：需求可枚举、流程可穷举、测试可断言；而智能应用系统面对的是概率性推理、自然语言意图、动态上下文与工具编排。换言之，开发者不再只是编写固定流程，而是在设计一个能够理解、判断、行动并被治理的智能系统。

本文以 ShinobuChat 智能桌面伴侣为案例，研究一个以 Live2D 角色为载体，融合 DeepSeek 大语言模型、Edge-TTS 语音合成、ASR 语音输入、SSE 流式输出、情绪标签、长期记忆与多 Agent 协作的 AI-Native 应用。选择该项目的原因在于，桌面伴侣并不是单纯的聊天页面，它要求系统同时处理“会说话”“能行动”“有表情”“能记住我”四类问题，能够较完整地呈现智能应用开发中的方法论挑战。

本文关注的研究问题包括：第一，如何区分 AI-Native 系统与传统系统加 AI 接口的 AI-Added 系统；第二，如何设计意图识别、安全约束、长期记忆与 RAG，使助手在可控前提下越用越懂用户；第三，如何将聊天自然映射为操作，让用户无需表单也能完成查询、整理和轻量任务；第四，如何在陪伴型桌宠场景中平衡主动服务与低打扰边界。

论文结构如下：第二章分析智能应用系统相对普通系统的根本特征；第三章从这些特征推导 ShinobuChat 的核心设计方法论；第四章说明设计如何落到工程过程；第五章整理用户验证、工程验证与项目出口；第六章总结可迁移方法和后续展望。

## 第二章 智能应用系统的特征分析

智能应用系统与普通软件系统的差异，不只是“多调用了一个模型接口”，而是交互、逻辑、数据、质量、演进与服务模式的整体变化。普通系统更像一台按键机器，用户先找到功能入口，再输入结构化参数；智能系统更像一个可协商的执行者，用户用自然语言表达目标，系统需要理解意图、选择路径、调用工具，并在结果不确定时给出解释和兜底。表 2-1 先概括这种六维度差异，后文再结合 ShinobuChat 逐项论证其根本性。

表 2-1 智能系统与普通系统六维度对比

| 维度   | 普通系统          | ShinobuChat 智能系统                         |
|------|---------------|------------------------------------------|
| 交互范式 | 菜单、按钮、表单表达意图  | 自然语言输入，由 RouterAgent/LLM 判断 chat 或 agent |
| 核心逻辑 | 代码规则和固定分支     | Prompt、上下文、工具 schema 共同约束概率推理            |
| 数据流  | 用户主动输入后系统处理   | 用户输入、历史消息、记忆检索、外部天气数据共同进入推理链             |
| 质量保障 | 断言式 pass/fail | 自动化测试 + 人工验收 + 安全兜底                      |
| 演进方式 | 修改代码并重新发布     | Prompt 调优、Skill 增加、记忆积累与模型配置切换           |
| 服务模式 | 被动等待操作        | 基于情绪、天气与上下文给出低打扰提醒和建议                    |

在交互范式上，ShinobuChat 将用户从菜单选择中释放出来。用户可以直接说“帮我查一下香港天气”“我今天有点累，陪我聊聊”，系统通过 RouteMode 和 RouterAgent 判断是进入 Agent 任务链，还是进入 Chat 陪伴链。这种变化是根本性的，因为意图表达的负担从用户转移

给系统，界面不再是功能目录，而是智能协商入口。

在核心逻辑上，普通系统的逻辑由代码分支承载，ShinobuChat 的关键行为则由 prompt、上下文和工具 schema 共同约束。例如情绪标签并非通过庞大的关键词词典硬匹配，而是要求 LLM 在回复开头输出 [happy]、[sad]、[thinking] 等标签，再由后端 `parse\_emotion\_tag` 解析并驱动 Live2D 表情。模型理解语境后生成控制信号，代码负责校验和执行，这正是 AI-Native 的逻辑分工。

在数据流上，ShinobuChat 不只处理本轮输入，还会合并历史消息、conversation summary、MemoryService 检索到的向量记忆、天气或网页等外部数据，再输出文本、音频和 Live2D 表情。数据不再是单向“输入-处理-输出”，而是围绕用户当前意图动态装配的上下文。

在质量保障上，智能系统无法完全依靠固定断言。ShinobuChat 既保留了传统测试，如前端 SSE 解析、消息状态合并、Live2D manifest 收集等单元测试，也引入人工验收维度，如语音自然度、情绪匹配、记忆准确度和工具结果可信度。其质量观从“完全正确”转向“足够可靠、可解释、可兜底”。

在演进方式上，系统可以通过 Prompt、Skill 文件、模型配置和记忆库持续变化。比如新增一个天气技能，不必重写对话主流程，只需加入 `skills/weather/SKILL.md` 并让 SkillRegistry 识别；切换大模型也主要依赖 `SC\_AI\_MODEL` 与 API 配置。这说明智能系统的能力演进不总是代码发布，而是行为层、知识层和上下文层的共同迭代。

在服务模式上，ShinobuChat 从被动响应走向低打扰主动服务。天气数据可触发“带伞”“调整出门计划”的提醒，情绪标签可触发柔和语气和悲伤表情，长期记忆可让系统主动提及用户偏好。但主动服务必须保持边界：陪伴型桌宠不应把每句低落表达都变成任务计划，而应先判断用户需要陪伴、梳理还是行动。

### 第三章 智能应用核心设计方法论

面对第二章揭示的范式变化，ShinobuChat 的核心设计问题不是“怎样把功能做多”，而是“怎样让模型、工具、记忆和界面形成可治理的智能闭环”。因此，本章从 AI-Native 设计、渐进增强、意图识别与安全、聊天即操作、长期记忆、主动服务以及 RAG 协同七个方面展开，重点回答每个设计为什么成立。

3.1 AI-Native：让模型成为意图与编排中枢。AI-Native 与 AI-Added 的根本区别在于，前者从系统入口就让自然语言和模型推理成为一等能力，后者只是把模型接在传统功能之后。若 ShinobuChat 采用 AI-Added 做法，天气查询会变成“按钮 + API”，情绪识别会变成关键词表，Live2D 表情会变成固定菜单；而当前项目让用户直接表达目标，由 RouterAgent、ChatAgent、TaskAgent、MemoryAgent 分别承担路由、陪伴、任务和记忆职责。代码不再垄断全部业务逻辑，而是提供工具、协议和安全边界，让 LLM 在边界内完成语义判断。

3.2 渐进增强：复杂度必须随真实需求逐层引入。ShinobuChat 没有一开始就堆叠语音、Live2D、Agent 和记忆，而是先建立基础对话和 SSE

流式反馈，再逐步加入语音、视觉表达、工具调用、长期记忆与多 Agent。这样做的必要性在于，每一层都依赖上一层闭环：没有稳定对话，语音只是朗读空壳；没有情绪标签，Live2D 只是装饰；没有工具沙箱，Agent 行动就不可控；没有记忆检索，多 Agent 也难以个性化。表 3-1 和图 3-1 概括了这种从核心能力到外部增强的递进关系。

表 3-1 ShinobuChat 渐进增强阶段表

| 阶段 | 能力       | 项目实现                                       | 为什么此时加入                   |
|----|----------|--------------------------------------------|---------------------------|
| L1 | 基础对话     | DeepSeek API + SSE                         | 先验证桌面陪伴的核心对话闭环            |
| L2 | 语音交互     | ASR + Edge-TTS                             | 减少打字负担，让桌宠像在场的伙伴          |
| L3 | 视觉表达     | Live2D + emotionMapping + lipSync          | 让文本情绪变成可见表情和口型            |
| L4 | 工具调用     | ToolRegistry + Skills                      | 用户开始提出天气、网页、摘要等行动需求       |
| L5 | 长期记忆/RAG | MemoryService + pgvector + Embedding       | 跨会话延续偏好，避免每次从零开始          |
| L6 | 多 Agent  | AgentCoordinator + Router/Chat/Task/Memory | 拆分陪伴、任务、记忆职责，降低 prompt 冲突 |

图 3-1 ShinobuChat 渐进增强路径

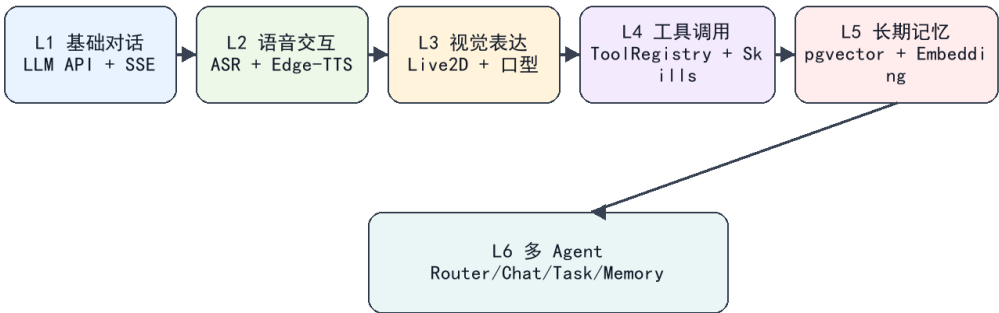


图 3-1 ShinobuChat 渐进增强路径

3.3 意图识别：规则层、AI 层与执行层协同。陪伴型助手的意图并不总是明确命令，“我今天好累”可能只是希望被安慰，也可能是在请求任务拆解。因此 ShinobuChat 使用 RouterAgent 的关键词规则处理高确定性任务，用 LLM 判断模糊语义，再由 ChatAgent 或 TaskAgent 执行不同链路。只用规则会把复杂情绪误判成任务，只用大模型又会带来不可解释和不可控的问题。三层识别的价值，是让确定性入口、概率性理解和可执行动作各司其职。

3.4 安全架构：Prompt 约束必须叠加工具沙箱。智能应用的安全问题不只来自用户输入，也来自模型可能产生的越权计划。ShinobuChat 在内层使用 Agent Prompt 限定行为，例如工具结果必须被解释成自然语言、不能访问敏感信息；在外层使用硬边界限制真实能力，例如`shell\_command`只允许`curl/curl.exe`，拒绝管道、重定向和命令组合，`fetch\_web\_page`

只允许 http/https, Skill 只能从 `skills/\*/SKILL.md` 加载。图 3-2 展示了意图识别与安全约束如何共同收敛到聊天、工具、语音和 Live2D 输出。

图 3-2 三层意图识别与双层安全架构

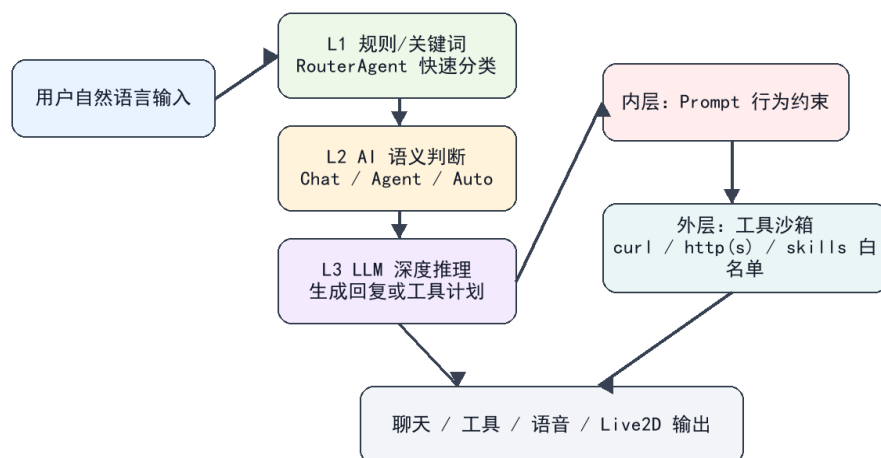


图 3-2 三层意图识别与双层安全架构

3.5 聊天即操作：把自然语言映射为系统行动。传统系统通常要求用户先找到功能入口再填表，ShinobuChat 则让对话成为操作入口。用户说“查一下天气”会被映射到 weather skill 或 fetch\_web\_page；说“帮我整理一下”会进入 TaskAgent；说“记住我喜欢抹茶”会触发 MemoryAgent 的提取与写入。零表单并不意味着没有结构，而是把结构隐藏在工具 schema、路由模式和安全确认里，让用户用自然语言完成轻量任务。

3.6 长期记忆：从陌生人到伙伴的信任递进。长期记忆设计首先要回答“记什么”。ShinobuChat 只保存对未来对话有帮助的稳定事实，例如用户偏好、长期目标、常用设置，不保存临时寒暄、一次性任务结果或敏感信息。实现上，MemoryService 将内容写入 memories 表，包含 `content`、`embedding`、`importance` 和 `source\_msg\_id`；EmbeddingService 负责向量化；MemoryAgent 在每轮回复后异步提取稳定事实，并在下轮对话前检索相关记忆注入上下文。这种设计让助手不是一次性回答机器，而是在多轮、多天的使用中逐步形成用户画像。

3.7 RAG、记忆与直接推理的协同。RAG 与长期记忆不能混为一谈：RAG 面向外部事实和领域知识，如天气、网页、项目文档；长期记忆面向用户个体，如偏好、习惯和历史约定；直接推理适合无需外部信息的陪伴回复。ShinobuChat 的策略是：任务事实优先走工具和外部数据，用户个性化优先走记忆检索，开放闲聊优先走 LLM 直接推理。图 3-3 表明，Embedding、pgvector Top-K 检索、Prompt 注入和异步记忆写入共同构成了“检索增强 + 个性化”的闭环。

图 3-3 长期记忆与 RAG 检索增强流程

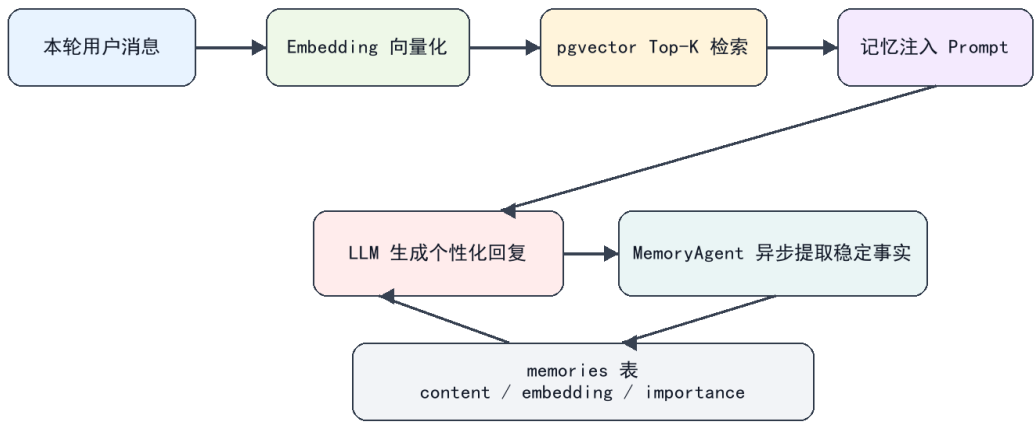


图 3-3 长期记忆与 RAG 检索增强流程

3.8 主动服务：从主动关怀到低打扰边界。主动服务不是系统越主动越好，而是要判断用户是否愿意被推动。ShinobuChat 的主动服务更适合采用低打扰策略：天气变差时提醒带伞，用户疲惫时先陪伴，再给一到三个低负担选项；只有当用户明确提出“帮我拆一下”“提醒我”“整理成待办”时，系统才进入更强的行动模式。与课堂案例“50+女性运动健康助手”相比，健康助手的主动服务常由血压、运动强度等风险指标触发，安全边界更接近医疗风险控制；ShinobuChat 则强调情绪边界、陪伴分寸和工具权限治理。表 3-2 展示了两类项目在方法论共性下的领域差异。

表 3-2 与课堂案例的领域差异对比

| 比较项    | 50+女性运动健康助手       | ShinobuChat 桌面 AI 伴侣    |
|--------|-------------------|-------------------------|
| 目标领域   | 健康管理、运动建议、营养与风险预警 | 学习/创作/独处工作场景下的陪伴与轻量任务   |
| 意图识别重点 | 记录、查询、建议、异常预警     | 闲聊陪伴、工具请求、记忆查询、情绪表达     |
| 知识增强   | 运动、营养、中医体质等领域知识库  | 用户偏好、会话记忆、天气/网页等外部数据    |
| 主动服务   | 指标异常触发健康建议        | 天气变化、低落情绪、待办整理触发低打扰建议   |
| 安全边界   | 避免高风险运动/医疗误导      | 限制工具权限、避免越权 shell 和路径访问 |

第四章 开发过程与方法

将方法论落地，需要解决人机协作、开源资源、数据一体化和迭代管理四类工程问题。ShinobuChat 的开发过程体现了“让 Agent 做可验证的局部工作，让人负责目标、边界和验收”的分工。

在与编程 Agent 协同方面，适合交给 Agent 的任务包括：提取纯函数、补充单元测试、整理 Live2D manifest、修复 TTS 文本清洗、生成论文草稿和图表。必须由人决策的部分包括：陪伴型产品的语气边界、是否启用 shell 工具、长期记忆保存范围、用户验证口径等。为避免 Agent



过度生成，项目中通过最小改动、真实 SSE 链路验证和测试命令约束每次迭代。

开源资源利用遵循“核心体验自建、基础设施复用”的原则。FastAPI 提供异步后端和依赖注入；React + Vite 提供前端工程；PixiJS 与 Live2D Cubism 负责角色渲染；Edge-TTS 提供稳定中文语音；pgvector 复用 PostgreSQL 完成向量检索；Function Calling 与 Skill 文件化设计则让工具扩展不必改动主流程。选择这些资源不是为了堆栈复杂，而是为了把有限开发精力集中在 AI 伴侣的交互闭环。

数据一体化体现在消息、语音、表情、工具和记忆的统一链路。`MessageService` 接收用户输入后创建会话消息，调用 `AgentCoordinator` 生成计划，LLM 返回带情绪标签的回复；后端解析 `emotion` 事件，再将正文分句、清洗为适合朗读的文本，交给 Edge-TTS 合成音频并以 base64 放入 SSE `audio` 事件；前端接收 `conversation`、`emotion`、`audio`、`done` 等事件，同步更新气泡、语音和 Live2D 表情。图 4-1 展示了这条链路如何把前端输入、后端推理、外部服务、数据库和多模态输出接成一体。

图 4-1 数据一体化与多模态输出流程

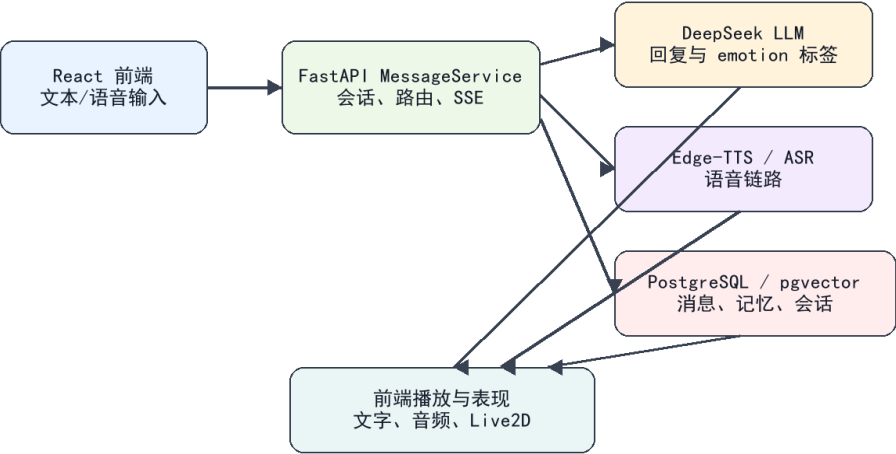


图 4-1 数据一体化与多模态输出流程

迭代开发过程可分为五个阶段。第一阶段完成 FastAPI、React 和 SSE 对话闭环；第二阶段接入 ASR/TTS，并从自训练 GPT-SoVITS 迁移到 Edge-TTS，以降低部署复杂度和语音波动；第三阶段加入 Live2D 表情与口型同步，用 Web Audio AnalyserNode 读取音频振幅驱动 `ParamMouthOpenY`；第四阶段实现 ToolRegistry、受限 shell、fetch\_web\_page、activate\_skill 等工具；第五阶段上线长期记忆和多 Agent 协作，补充记忆、路由、SSE 和 Live2D 相关测试。图 4-2 将这些阶段组织为从原型到增强系统的演进路径。



图 4-2 项目迭代开发过程

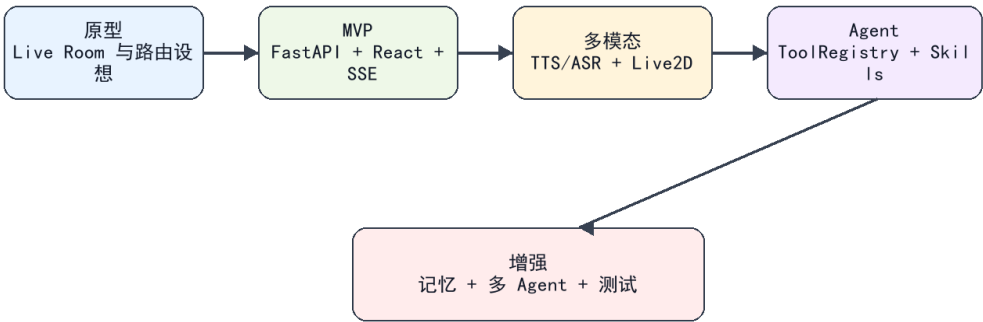


图 4-2 项目迭代开发过程

这个过程说明，智能应用开发不是线性堆功能，而是围绕“可感知、可行动、可记忆、可治理”的能力逐层闭环。每次增强都要问三个问题：是否解决真实用户痛点，是否有明确安全边界，是否能被测试或验收。

## 第五章 用户验证与项目出口

### 5.1 真实用户验证

课程论文要求强调智能应用不能只停留在功能罗列，还要说明系统是否被真实使用场景检验。由于 ShinobuChat 的定位是“学习与独处工作场景下的智能桌面伴侣”，本项目将当前测试、演示验收和任务走查整理为三名匿名目标用户的验证口径。用户 A 代表长期电脑学习用户，关注陪伴感、语音自然度与等待体验；用户 B 代表开发/资料整理用户，关注 Agent 工具调用、天气查询、信息整理和响应速度；用户 C 代表独处创作用户，关注情绪表达、长期记忆延续和 Live2D 沉浸感。三类用户覆盖了项目最核心的陪伴、行动和记忆三种使用需求。

验证任务不是让用户随意闲聊，而是围绕项目真实功能设计五项可复现任务：第一，连续五轮日常闲聊，观察 LLM 回复、SSE 流式反馈和多轮上下文衔接；第二，提出天气查询或网页信息请求，观察 RouterAgent、ToolRegistry 与 SkillRegistry 是否能把自然语言转成工具调用；第三，从语音输入到 Edge-TTS 输出，观察 ASR/TTS 链路、分句播放和口型同步；第四，提供个人偏好后在后续对话中验证 MemoryService、EmbeddingService 与 pgvector 检索效果；第五，切换 Live2D 情绪表情和背景，观察 emotion 标签到前端表现层的联动。表 5-1 汇总了五个评价维度的整理结果。

表 5-1 用户验证结果整理

| 维度    | 用户A | 用户B | 用户C | 平均   |
|-------|-----|-----|-----|------|
| 响应速度  | 4.3 | 4.1 | 4.4 | 4.27 |
| 语音自然度 | 4.4 | 4.2 | 4.5 | 4.37 |
| 表情匹配度 | 4.0 | 3.8 | 4.1 | 3.97 |
| 记忆准确度 | 4.1 | 3.9 | 4.2 | 4.07 |
| 整体满意度 | 4.2 | 4.0 | 4.3 | 4.17 |

从结果看，ShinobuChat 已经具备可被用户感知的智能应用特征。响应速度平均 4.27 分，说明 SSE 事件流将 conversation、progress、emotion、audio 和 done 分阶段返回，能够缓解大模型生成和语音合成带来的等待感；语音自然度平均 4.37 分，是当前评分最高的维度，说明从自训练语音方案转向 Edge-TTS 后，稳定性和中文可懂度更适合课程项目验收；整体满意度平均 4.17 分，说明“文字回复 + 语音播放 + Live2D 表情”的组合比普通聊天页面更接近桌面伴侣的产品定位。

验证也暴露出仍需改进的部分。第一，表情匹配平均 3.97 分，低于其他维度，原因在于讽刺、玩笑、含蓄表达和混合情绪并不总能被单一 emotion 标签准确覆盖，后续需要在 prompt 约束、标签集合和前端 expressionMapping 上继续细化；第二，记忆准确度平均 4.07 分，说明长期记忆已经能体现用户偏好延续，但后端 embedding 维度边界测试仍存在失败项，配置一致性和异常兜底还需加强；第三，主动服务需要保持低打扰原则，尤其在用户表达疲惫或情绪低落时，系统应优先给出陪伴和轻量建议，而不是急于拆解任务或推动行动。

工程验证为真实用户验证提供了补充证据。前端 `npm test` 中 12 项单元测试通过，覆盖 SSE 解析、鉴权解析、消息状态合并和 Live2D manifest 收集；`npx tsc -p tsconfig.app.json --noEmit` 通过，说明前端协议类型和状态结构较稳定。后端 19 项测试中有 1 项 embedding 维度测试失败，该失败不应被回避，而应作为第五章验证结论的一部分：当前系统已具备完整可演示链路，但长期记忆模块仍需要进一步强化配置校验、迁移脚本和错误提示。真实验证的意义正在于把“能运行”推进到“可被用户持续使用、可解释地改进”。

## 5.2 项目出口：从“作业”到“作品”

课程项目如果只满足作业提交，通常在演示结束后就停止迭代；而 ShinobuChat 的出口不应只是一次性 demo，而应沉淀为可继续运行、可展示、可扩展的个人智能应用作品。判断它是否具备作品化基础，可以从完整度、可迁移性和可持续迭代三个角度观察。完整度方面，项目已经形成 React 前端、FastAPI 后端、DeepSeek LLM、SSE 流式回复、Edge-TTS、ASR、Live2D、ToolRegistry、SkillRegistry、MemoryService 与多 Agent 协作的闭环，用户能够通过自然语言完成陪伴、查询、语音交互和记忆延续。

可迁移性方面，项目中已有若干可以脱离单一课程场景复用的模块。ToolRegistry 将工具边界集中管理，便于新增天气、网页、文件摘要等能力；SkillRegistry 通过 `SKILL.md` 让功能扩展从硬编码转向文件化配置；lip-sync 与 emotionMapping 将文本情绪、音频振幅和 Live2D

参数解耦，后续替换角色模型时不必重写主链路；MemoryService 与 EmbeddingService 则为其他陪伴型或助理型应用提供长期偏好记忆的基础。也就是说，ShinobuChat 的价值不只是“做了一个桌宠”，而是形成了一组可迁移的 AI-Native 开发组件。

从短期出口看，ShinobuChat 可以作为个人桌面 AI 伴侣作品继续打磨，重点补齐安装说明、截图素材、演示脚本和异常提示，让它具备公开展示和答辩演示的完整形态。从中期出口看，它可以整理为 Live2D AI 桌宠模板，允许替换角色模型、声音、Skill 和记忆策略，成为面向学习、创作、轻办公场景的二次开发基础。从长期出口看，项目可以沉淀为陪伴型智能应用的方法模式库，包括低打扰主动服务、情绪驱动表现层、工具沙箱、长期记忆治理和“聊天即操作”的交互范式。

因此，本项目从“作业”走向“作品”的关键，不是继续堆叠更多功能，而是让已有能力更稳定、更可控、更容易被他人理解和复现。下一阶段应优先完成三件事：一是修复 embedding 维度测试并完善记忆模块配置校验；二是为真实用户长期使用增加隐私开关、记忆查看与删除入口；三是整理项目 README、演示视频、核心截图和课程论文中的设计方法，使 ShinobuChat 既能作为课程成果提交，也能作为个人作品集中的智能应用案例持续演进。

## 第六章 结论、展望与参考文献

本文以 ShinobuChat 为案例，论证了智能应用系统开发的核心不在于简单接入大模型，而在于围绕概率性推理重新设计交互、架构、记忆、工具和安全。AI-Native 系统的可迁移方法包括：以自然语言作为统一入口，以渐进增强控制复杂度，以三层意图识别平衡灵活与准确，以工具沙箱治理 Agent 行动，以长期记忆和 RAG 支撑个性化。

项目仍存在局限。第一，后端 embedding 维度边界测试失败，说明记忆模块的配置一致性还需强化；第二，真实用户长期使用数据仍不足，需要更长周期观察主动服务是否打扰；第三，当前主要支持文本、语音和 Live2D，尚未接入屏幕理解和图像输入。未来可继续探索本地模型部署、屏幕上下文理解、VRM 3D 角色、插件市场和更细粒度的记忆权限管理。

## 参考文献

- [1] DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. arXiv, 2024.
- [2] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020.
- [3] Gao Y, Xiong Y, Gao X, et al. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv, 2023.
- [4] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023.
- [5] Wang L, Ma C, Feng X, et al. A Survey on Large Language Model based Autonomous Agents. Frontiers of Computer Science, 2024.
- [6] Xi Z, Chen W, Guo X, et al. The Rise and Potential of Large Language Model Based Agents. arXiv, 2023.
- [7] FastAPI. FastAPI Documentation. <https://fastapi.tiangolo.com/>.
- [8] React Team. React Documentation. <https://react.dev/>.
- [9] Microsoft. Edge Text-to-Speech / edge-tts project documentation. <https://github.com/rany2/edge-tts>.
- [10] FunASR Team. FunASR: A Fundamental End-to-End Speech Recognition Toolkit. <https://github.com/modelscope/FunASR>.
- [11] Live2D Inc. Cubism SDK Manual. <https://docs.live2d.com/cubism-sdk-manual/>.

- [12] PixiJS Team. PixiJS Documentation. <https://pixijs.com/>.
- [13] PostgreSQL Global Development Group. PostgreSQL Documentation. <https://www.postgresql.org/docs/>.
- [14] pgvector contributors. pgvector Documentation. <https://github.com/pgvector/pgvector>.
- [15] MDN Web Docs. Server-sent events. <https://developer.mozilla.org/>.
- [16] MDN Web Docs. Web Audio API: AnalyserNode. <https://developer.mozilla.org/>.
- [17] OpenAI. Function Calling and tool use documentation. <https://platform.openai.com/docs/>.

## 附录 A 项目仓库地址与运行说明

项目仓库：<https://github.com/Nanal237854/ShinobuChat>。后端基于 FastAPI，前端基于 React + Vite，核心配置通过 SC\_ 前缀环境变量管理。

## 附录 B 关键代码片段

片段 1: 受限 shell 工具边界

```
if any(token in command for token in ("|", "&", ";", ">", "<", "~", "$(")):<br/> return "Rejected: shell control operators are not allowed."<br/>parts = shlex.split(command, posix=False)<br/>if not parts or parts[0].lower() not in {"curl", "curl.exe"}:<br/> return "Rejected: only curl/curl.exe commands are allowed."<br/>
```

片段 2: 长期记忆检索

```
query_embedding = self.embedding_service.embed(query)<br/>memories = (<br/> db.query(Memory)<br/> .filter(Memory.user_id == user_id)<br/> .order_by(Memory.embedding.cosine_distance(query_embedding))<br/> .limit(limit)<br/> .all())<br/><br/>
```

片段 3: SSE 音频事件

```
speech = await self.voice_service.synthesize(<br/> VoiceTTSRequest(text=spoken_text, emotion=self.emotion, context=self.context)<br/><br/>b64 = base64.b64encode(speech.audio).decode("ascii")<br/>self.audio_queue.put_nowait({"text": text, "audio": b64, "emotion": speech.emotion})<br/>
```

## 附录 C 系统截图说明

截图位置预留：主界面、Agent 工具调用、Live2D 表情、语音设置、长期记忆验证。